

SESIÓN

15 | Seguridad del sitio web

UNIDAD DE APRENDIZAJE 4

- Manejo de la seguridad por roles
- Reglas de acceso a las páginas y carpetas del sitio
- Formulario Login (autenticación por cookie)
- Caducidad de la sesión

/ INTRODUCCIÓN

- En las aplicaciones web se maneja la **seguridad del sitio**: qué usuarios pueden ingresar (con sus credenciales) y qué pueden hacer una vez dentro, según su **rol**.
- También importa la **caducidad de la sesión**: cuánto tiempo permanece activa tras la última interacción del usuario, algo típico en aplicaciones bancarias, financieras o de gestión.
- Hoy aprenderemos a: crear **roles y usuarios**, definir el **acceso por página o carpeta** según el rol, armar el **menú** según el usuario, implementar el **formulario Login** y manejar la **caducidad de la sesión** con JavaScript.

Del stack antiguo (Web Forms, **aspnet_regsql.exe**, Membership/RoleProvider, Web.config por carpeta, SiteMap y control Login) pasamos al actual: **ASP.NET Core Razor Pages**, **.NET 10**, autenticación por **cookie**, autorización por **políticas y roles** y recorte del menú por rol, como en las sesiones anteriores.

/ MANEJO DE LA SEGURIDAD POR ROLES

/ MANEJO DE LA SEGURIDAD POR ROLES

¿Qué es la seguridad basada en roles?

- La seguridad basada en roles (**Role-Based Access Control**) asigna permisos a los usuarios según su **función o rol** dentro de la organización, no de forma individual.
- Los usuarios se agrupan en roles (por ejemplo **Administrador** y **Operador**), y a cada rol se le otorgan los permisos que necesita para su trabajo.
- Ventajas: **simplifica** la administración de permisos (un cambio en el rol aplica a todos sus usuarios) y **reduce riesgos**, limitando el acceso solo a lo necesario.

En nuestro caso de uso hay dos roles: **Administrador** (acceso total) y **Operador** (solo consultas). Las páginas se protegen según el rol.

/ MANEJO DE LA SEGURIDAD POR ROLES

Autenticación y Autorización

AUTENTICACIÓN

- Proceso por el que se **comprueba la identidad** de quien intenta ingresar.
- Requiere **credenciales** (usuario y contraseña). Si son correctas, se acepta que es quien dice ser.

AUTORIZACIÓN

- Proceso que determina **qué puede hacer** ese usuario ya autenticado.
- Según su **rol**, se le habilita el acceso a ciertas páginas, carpetas u opciones del menú.

Primero se autentica (**quién es**), luego se autoriza (**qué puede hacer**). En ASP.NET Core esto se declara en **Program.cs**: `UseAuthentication()` antes de `UseAuthorization()`.

/ MANEJO DE LA SEGURIDAD POR ROLES

Autenticación por formulario y cookie

- El usuario ingresa sus credenciales en un **formulario Login**. La aplicación las valida y, si son correctas, crea una **cookie de autenticación** cifrada en el navegador.
- En las siguientes peticiones esa cookie identifica al usuario, sin volver a pedir credenciales, hasta que caduca o cierra sesión.
- En ASP.NET Core la identidad se guarda como **claims**: el nombre del usuario y un claim por cada **rol**. La autorización se basa en esos claims.

Equivalencia: el antiguo `<authentication mode="Forms">` del Web.config se reemplaza por `AddAuthentication().AddCookie(...)`; el **roleManager** y el **membership** ya no se usan (los roles viajan como claims en la cookie).

/ MANEJO DE LA SEGURIDAD POR ROLES

Objetos de seguridad en la base de datos

- En Web Forms los objetos de seguridad se creaban con el utilitario **aspnet_regsql.exe** (tablas aspnet_Membership, aspnet_Roles, aspnet_Users...). Ese proveedor **no existe** en ASP.NET Core.
- Aquí usamos un **esquema propio y sencillo** sobre la base VentasLeon: **Tb_Rol**, **Tb_Usuario** y la tabla puente **Tb_Usuario_Rol** (un usuario puede tener uno o varios roles).
- La contraseña se guarda como **hash**, nunca en texto plano.

Tb_Rol	Tb_Usuario	Tb_Usuario_Rol
Cod_rol (PK)	Cod_usu (PK)	Cod_usu (PK, FK)
Nom_rol	Login	Cod_rol (PK, FK)
	Clave (hash)	
	Nombre / Email	
	Est_usu	

/ MANEJO DE LA SEGURIDAD POR ROLES

Creando roles y usuarios desde la capa de datos

VentasRepositorio · CrearUsuarioAsync

```
// capa de datos: crea el usuario, hashea la clave y lo asigna a un rol
public async Task<bool> CrearUsuarioAsync(UsuarioNuevo datos)
{
    if (await _ctx.Usuarios.AnyAsync(u => u.Login == datos.Login))
        return false; // login repetido

    var u = new Usuario { Login = datos.Login, Nombre = datos.Nombre,
                          EstUsu = 1 };
    // la clave se guarda como hash (PasswordHasher)
    u.Clave = _hasher.HashPassword(u, datos.Clave);
    u.Roles.Add(new UsuarioRol { CodRol = datos.CodRol });

    _ctx.Usuarios.Add(u);
    await _ctx.SaveChangesAsync();
    return true;
}
```

VentasRepositorio · ValidarCredencialesAsync

```
// valida usuario + clave contra el hash guardado
public async Task<UsuarioAutenticado?> ValidarCredencialesAsync(
    string login, string clave)
{
    var u = await _ctx.Usuarios
        .Include(x => x.Roles).ThenInclude(r => r.Rol)
        .FirstOrDefaultAsync(x => x.Login == login && x.EstUsu == 1);
    if (u == null) return null;

    var ok = _hasher.VerifyHashedPassword(u, u.Clave, clave);
    if (ok == PasswordVerificationResult.Failed) return null;
    // devuelve el usuario con la lista de sus roles
    return new UsuarioAutenticado { ... };
}
```


/ MANEJO DE LA SEGURIDAD POR ROLES

Configurando autenticación y autorización en Program.cs

Program.cs

```
// autenticacion por cookie (reemplaza <authentication mode="Forms">)
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(o => {
        o.LoginPath = "/Login";
        o.AccessDeniedPath = "/AccesoDenegado";
        o.ExpireTimeSpan = TimeSpan.FromMinutes(minutosSesion);
        o.SlidingExpiration = true;    // se renueva con la actividad
    });

// politicas por rol (reemplazan al roleManager del Web.config)
builder.Services.AddAuthorization(o => {
    o.AddPolicy("EsAdministrador", p => p.RequireRole("Administrador"));
    o.AddPolicy("AdminUOperador",
        p => p.RequireRole("Administrador", "Operador"));
});
```

/ LABORATORIO

/ LABORATORIO 1

Implementar los objetos de seguridad y crear roles y usuarios

- Crear en la base VentasLeon las tablas **Tb_Rol**, **Tb_Usuario** y **Tb_Usuario_Rol** (script **VentasLeon_S15.sql**).
- En la capa de datos, implementar los métodos para **crear roles**, **dar de alta usuarios** (con la clave hasheada) y **asignarlos a un rol**.
- Sembrar los usuarios de prueba: **admin** (Administrador) y **operador** (Operador), y gestionarlos desde la página **Seguridad**.

/ REGLAS DE ACCESO A LAS PÁGINAS DEL SITIO

/ REGLAS DE ACCESO A LAS PÁGINAS DEL SITIO

- En Web Forms cada carpeta llevaba su propio **Web.config** con reglas `<allow>` / `<deny>` por rol.
- En Razor Pages la autorización se centraliza en **Program.cs** con **convenciones** (`AuthorizeFolder` / `AuthorizePage`) y **políticas** por rol.

Web Forms (Web.config por carpeta)	ASP.NET Core (Program.cs)
<code><deny users="?"></code> en la raíz	<code>Conventions.AuthorizeFolder("/")</code>
Mantenimientos → <code>allow roles="Administrador"</code>	<code>AuthorizePage("/Proveedores", "EsAdministrador")</code>
Consultas → <code>allow roles="Administrador,Operador"</code>	<code>AuthorizePage("/Reporte", "AdminUOperador")</code>
Seguridad → <code>allow roles="Administrador"</code>	<code>AuthorizeFolder("/Seguridad", "EsAdministrador")</code>
Login/Logout accesibles a todos	<code>AllowAnonymousToPage("/Login")</code>

/ REGLAS DE ACCESO A LAS PÁGINAS DEL SITIO

El menú según el rol del usuario

- En Web Forms el menú se armaba con **Web.sitemap** + **SiteMapDataSource** + control **Menu**, y el `securityTrimming` ocultaba las opciones no permitidas.
- En Razor Pages el mismo efecto se logra en el **_Layout**, mostrando cada opción con `User.IsInRole(...)`.

_Layout.cshtml

```
<!-- _Layout.cshtml: el menu se recorta segun el rol del usuario -->
@if (User.IsInRole("Administrador"))
{
    <li><a asp-page="/Proveedores">Mantenimientos</a></li>
    <li><a asp-page="/Seguridad/Index">Seguridad</a></li>
}
@if (User.IsInRole("Administrador") || User.IsInRole("Operador"))
{
    <li><a asp-page="/Reporte">Consultas</a></li>
}
```

/ LABORATORIO

/ LABORATORIO 2

Seguridad por página y acceso mediante el formulario Login

- Aplicar las **convenciones** de autorización a cada página según su rol (Mantenimientos y Transacciones solo Administrador; Consultas Administrador u Operador).
- Implementar el **formulario Login** y el recorte del **menú** por rol.
- Probar el acceso iniciando sesión como **admin** y como **operador**, verificando qué opciones ve cada uno.

/ EL FORMULARIO LOGIN

/ EL FORMULARIO LOGIN

- En Web Forms se usaba el control **Login**, que ya venía «pre programado». En Razor Pages creamos una página **Login** con un formulario y validamos las credenciales en el code-behind.
- Si son correctas, se crea la cookie con `SignInAsync`, incluyendo el **nombre** y un **claim de rol** por cada rol del usuario.

Login.cshtml.cs

```
// valida y firma la cookie con el nombre y los roles del usuario
var u = await _repo.ValidarCredencialesAsync(Entrada.Login, Entrada.Clave);
if (u == null) { Mensaje = "Usuario o contraseña incorrectos."; return Page(); }

var claims = new List<Claim> { new(ClaimTypes.Name, u.Login) };
foreach (var rol in u.Roles)
    claims.Add(new Claim(ClaimTypes.Role, rol)); // un claim por rol

await HttpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme,
    new ClaimsPrincipal(new ClaimsIdentity(claims, ...)));
```

/ EL FORMULARIO LOGIN

Usuario actual y cierre de sesión

- Los controles **LoginName**, **LoginView** y **LoginStatus** de Web Forms se reemplazan por el acceso directo a `User.Identity.Name` en el **_Layout**.
- Al colocarlo en la página maestra (`_Layout`), el nombre del usuario aparece en **todas** las páginas del sitio.
- El cierre de sesión (Logout) borra la cookie con `SignOutAsync` y vuelve al formulario Login.

La barra superior muestra **Usuario: (nombre)** y un botón **Salir**, visibles en todo el aplicativo, tal como el control `LoginName` del material antiguo.

/ MANEJANDO LA CADUCIDAD DE LA SESIÓN

/ MANEJANDO LA CADUCIDAD DE LA SESIÓN

¿Qué es la caducidad de la sesión?

- Cuando el usuario deja de interactuar con la aplicación por un tiempo, esta le avisa que la sesión va a terminar por **inactividad** y, si no reacciona, la cierra.
- Se usa mucho en aplicaciones **bancarias, financieras o de gestión**, para preservar la confidencialidad de la información.

La caducidad tiene dos partes: la del **servidor** (la cookie expira tras cierto tiempo) y la del **cliente** (JavaScript que avisa y redirige al Login).

/ MANEJANDO LA CADUCIDAD DE LA SESIÓN

Configuración (servidor) y aviso de inactividad (cliente)

Servidor · cookie

```
// appsettings.json
"Seguridad": { "AdvertenciaSesionMin": 1,
               "TimeoutSesionMin": 2 }

// Program.cs (servidor)
o.ExpireTimeSpan = TimeSpan.FromMinutes(2);
o.SlidingExpiration = true;
```

Cliente · JavaScript

```
// sesion-caducidad.js (cliente): cuenta la inactividad real
function iniciarControlSesion(advMin, timeoutMin) {
  function programar() {
    setTimeout(() => alert("Tu sesion va a expirar..."),
               advMin*60*1000);           // aviso
    setTimeout(() => { window.location = "/Logout"; },
               timeoutMin*60*1000);       // cierre
  }
  // cualquier actividad reinicia el conteo
  ["mousemove", "keydown", "scroll"].forEach(
    e => document.addEventListener(e, programar));
  programar();
}
```

/ LABORATORIO

/ LABORATORIO 3

Implementar la caducidad de la sesión web

- Configurar en **appsettings.json** los minutos de advertencia y de cierre, y en **Program.cs** el `ExpiresTimeSpan` de la cookie.
- Agregar en el **_Layout** la función JavaScript de control de inactividad, para que se herede a todas las páginas.
- Probar: dejar la aplicación inactiva y verificar el **aviso** y luego el **cierre** de sesión con retorno al Login.

/ CONCLUSIONES

/ CONCLUSIONES

- Asegurar el sitio web manejando **roles** es la forma más eficiente de controlar quién entra y qué puede hacer.
- En ASP.NET Core la seguridad se resuelve con **autenticación por cookie, autorización por políticas y roles y convenciones** por página, sin el proveedor Membership ni el SiteMap del stack antiguo.
- La **caducidad de la sesión** combina la expiración de la cookie en el servidor con un aviso de inactividad en el cliente.

/ BIBLIOGRAFÍA

Páginas web y otros recursos

- Microsoft Learn: «Introducción a la autorización en ASP.NET Core» y «Autenticación por cookies sin ASP.NET Core Identity».
- Microsoft Learn: «Autorización de páginas de Razor mediante convenciones (AuthorizeFolder / AuthorizePage)».
- Microsoft Learn: «PasswordHasher: hash de contraseñas en ASP.NET Core».
- OSCAR'S CODE / GEEKS.MS: artículos sobre advertencia y detección de expiración de sesión (adaptados al enfoque actual con JavaScript).